

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 07

NOMBRE COMPLETO: Pablo Sebastian Noyola Torres

Nº de Cuenta: 320023213

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027 - 1

FECHA DE ENTREGA LÍMITE: 10 de octubre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Añadir una luz de faro (spotlight) frontal al rover con jerarquía suyo que ilumine hacia adelante y al avanzar o retroceder el Rover, la luz siga unida a él.

Capturas de código:

```
57  //Sustituir por sus propios Rovers Con jerarquía
58  Model Rover_cuerpo;
59  Model Rover_basebrazo;
60  Model Rover_partebrazo1;
61  Model Rover_partebrazo2;
62  Model Rover_partebrazo3;
63  Model Rover_partebrazo4;
64  Model Rover_pata1;
65  Model Rover_llanta1;
66  Model Rover_pata2;
67  Model Rover_llanta2;
68  Model Rover_pata3;
69  Model Rover_llanta3;
70  Model Rover_pata4;
71  Model Rover_llanta4;
72  Model Rover_pata5_parte1;
73  Model Rover_pata5_parte2;
74  Model Rover_llanta5;
75  Model Rover_pata6_parte1;
76  Model Rover_pata6_parte2;
77  Model Rover_llanta6;
```

```
467     Rover_cuerpo = Model();
468     Rover_cuerpo.LoadModel("Models/Rover_cuerpo.obj");
469     Rover_basebrazo = Model();
470     Rover_basebrazo.LoadModel("Models/Rover_basebrazo.obj");
471     Rover_partebrazo1 = Model();
472     Rover_partebrazo1.LoadModel("Models/Rover_partebrazo1.obj");
473     Rover_partebrazo2 = Model();
474     Rover_partebrazo2.LoadModel("Models/Rover_partebrazo2.obj");
475     Rover_partebrazo3 = Model();
476     Rover_partebrazo3.LoadModel("Models/Rover_partebrazo3.obj");
477     Rover_partebrazo4 = Model();
478     Rover_partebrazo4.LoadModel("Models/Rover_partebrazo4.obj");
479     Rover_pata1 = Model();
480     Rover_pata1.LoadModel("Models/Rover_pata1.obj");
481     Rover_llanta1 = Model();
482     Rover_llanta1.LoadModel("Models/Rover_llanta1.obj");
483     Rover_pata2 = Model();
484     Rover_pata2.LoadModel("Models/Rover_pata2.obj");
485     Rover_llanta2 = Model();
486     Rover_llanta2.LoadModel("Models/Rover_llanta2.obj");
487     Rover_llanta3 = Model();
488     Rover_llanta3.LoadModel("Models/Rover_llanta3.obj");
489     Rover_llanta4 = Model();
490     Rover_llanta4.LoadModel("Models/Rover_llanta4.obj");
491     Rover_pata3 = Model();
492     Rover_pata3.LoadModel("Models/Rover_pata3.obj");
493     Rover_pata4 = Model();
494     Rover_pata4.LoadModel("Models/Rover_pata4.obj");
495     Rover_pata5_parte1 = Model();
496     Rover_pata5_parte1.LoadModel("Models/Rover_pata5_parte1.obj");
497     Rover_pata5_parte2 = Model();
498     Rover_pata5_parte2.LoadModel("Models/Rover_pata5_parte2.obj");
499     Rover_llanta5 = Model();
500     Rover_llanta5.LoadModel("Models/Rover_llanta5.obj");
501     Rover_pata6_parte1 = Model();
502     Rover_pata6_parte1.LoadModel("Models/Rover_pata6_parte1.obj");
503     Rover_pata6_parte2 = Model();
504     Rover_pata6_parte2.LoadModel("Models/Rover_pata6_parte2.obj");
505     Rover_llanta6 = Model();
506     Rover_llanta6.LoadModel("Models/Rover_llanta6.obj");
```

```
559     // luz de Faro para el Rover
560     spotLights[2] = SpotLight(
561         1.0f, 1.0f, 0.9f,
562         1.0f, 4.0f,
563         0.0f, 0.0f, 0.0f,
564         0.0f, 0.0f, 1.0f,
565         1.0f, 0.01f, 0.001f,
566         35.0f
567     );
568     spotLightCount++;
```

```
675     //Rover: Sustituido
676     // Cuerpo principal
677     color = glm::vec3(0.549f, 0.412f, 0.275f);
678     modelCuerpo = glm::mat4(1.0);
679     modelCuerpo = glm::translate(modelCuerpo, glm::vec3(0.0f + mainWindow.getmuevex(), 3.30f, -15.0));
680
681     // Para faro
682     glm::vec4 posicionLocalFaro = glm::vec4(1.5f, 0.5f, 0.0f, 1.0f);
683     glm::vec3 posicionMundoFaro = glm::vec3(modelCuerpo * posicionLocalFaro);
684     glm::vec4 direccionLocalFaro = glm::vec4(1.0f, 0.0f, 0.0f, 0.0f);
685     glm::vec3 direccionMundoFaro = glm::vec3(modelCuerpo * direccionLocalFaro);
686     direccionMundoFaro = glm::normalize(direccionMundoFaro);
687
688     spotLights[2].SetFlash(posicionMundoFaro, direccionMundoFaro);
689
690     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
691     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelCuerpo));
692     Rover_cuerpo.RenderModel();
693     color = glm::vec3(0.0f, 0.0f, 0.0f);
694
```

Capturas de ejecución:




PracticaXX:Nombre de la practica


PracticaXX:Nombre de la practica


PracticaXX:Nombre de la practica
Gen Grevious

En este ejercicio se implementa la integración de una luz de faro frontal (spotlight) con jerarquía para el rover, utilizando un enfoque matricial para que la iluminación acompañe de manera precisa al vehículo durante sus desplazamientos. Primero, se definen y cargan las mallas y componentes que estructuran el rover, incluyendo el cuerpo principal, base de brazo y llantas. Asimismo, se inicializa la configuración de la luz en el arreglo global utilizando el índice 2 (spotLights[2]) con sus respectivas propiedades de color, intensidad y ángulo de apertura. En la etapa de renderizado, la matriz de transformación del cuerpo (modelCuerpo) gestiona la posición del vehículo aplicando una traslación dinámica en el eje X mediante la entrada del usuario (mainWindow.getmuevex()). Finalmente, para establecer la jerarquía, se multiplican las coordenadas y direcciones locales del faro (posicionLocalFaro y direccionLocalFaro) por la matriz modelCuerpo, obteniendo así los vectores en el espacio del mundo que aseguran que el haz de luz permanezca unido al rover al avanzar o retroceder en la escena.

Añadir una luz de faro (spotlight) al avión con jerarquía suyo que ilumine hacia abajo y al subir y bajar el avión, la luz siga unida a él.

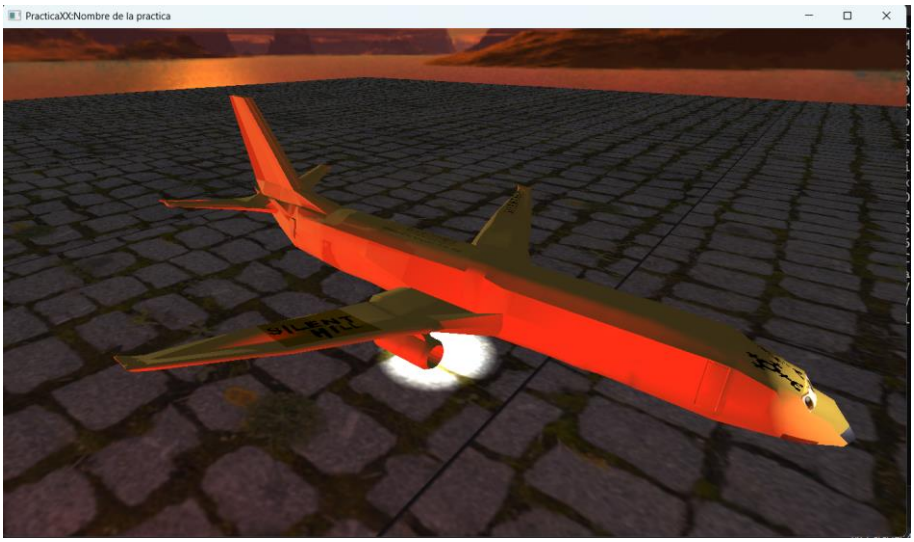
Capturas de código:

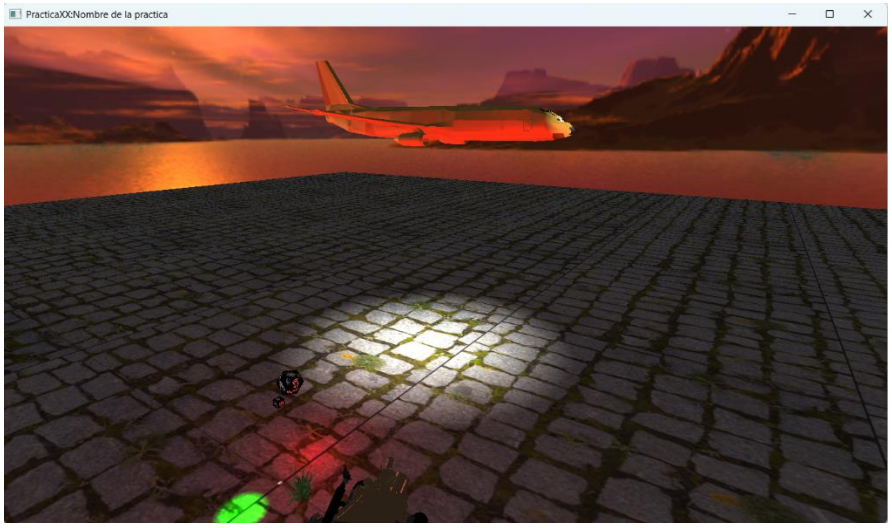
```
77    //Añadir modelo del avión.
78    Model Avion_M;
```

```
569     spotLights[3] = SpotLight(
570         1.0f, 1.0f, 0.9f,
571         1.0f, 4.0f,
572         0.0f, 0.0f, 0.0f,
573         0.0f, 0.0f, 1.0f,
574         1.0f, 0.01f, 0.001f,
575         35.0f
576     );
577     spotLightCount++;
```

```
904     //Añadir modelo del avión
905     // Avion----------
906     // Faro 1
907     model = glm::mat4(1.0f);
908     model = glm::translate(model, glm::vec3(-40.0f, 5.0f + mainWindow.getmueveyavion(), 6.0f + mainWindow.getmuevexavion()));
909     model = glm::scale(model, glm::vec3(8.0f, 8.0f, 8.0f));
910     glm::vec4 posicionLocalFaroAvion = glm::vec4(0.0f, -0.06f, 0.0f, 1.0f);
911     glm::vec3 posicionMundoFaroAvion = glm::vec3(model * posicionLocalFaroAvion);
912     glm::vec4 direccionLocalFaroAvion = glm::vec4(0.0f, -1.0, 0.0f, 0.0f);
913     glm::vec3 direccionMundoFaroAvion = glm::vec3(model * direccionLocalFaroAvion);
914     direccionMundoFaroAvion = glm::normalize(direccionMundoFaroAvion);
915     spotLights[3].SetFlash(posicionMundoFaroAvion, direccionMundoFaroAvion);
```

Capturas de ejecución:


PracticaXX:Nombre de la practica


PracticaXX:Nombre de la practica


PracticaXX:Nombre de la practica

En este ejercicio se implementa la incorporación de una luz de faro (spotlight) descendente con jerarquía para el avión, asegurando que la iluminación hacia abajo permanezca vinculada al modelo durante sus desplazamientos verticales de ascenso y descenso. Primero, se define el modelo del avión (Avion_M) y se inicializa la configuración de la luz en el arreglo global en la posición 3 (spotLights[3]), incrementando el contador de luces spotLightCount++. Durante la renderización, la matriz de transformación (model) controla la posición espacial del avión aplicando una traslación vertical controlada por la función mainWindow.getmueveyavion() y una escala uniforme. Por último, se transforman las coordenadas y direcciones locales del faro (posicionLocalFaroAvion y direccionLocalFaroAvion) multiplicándolas por la matriz model, lo que permite calcular las coordenadas del mundo y lograr que el haz de luz siga al avión cada vez que sube o baja en la escena.

Añadir una luz de faro (spotlight) al avión con jerarquía suyo que ilumine hacia adelante y al avanzar o retroceder el avión, la luz siga unida a él.

Capturas de código:

shader_light.frag | CommonValues.h

```
1   #version 330
2
3   in vec4 vCol;
4   in vec2 TexCoord;
5   in vec3 Normal;
6   in vec3 FragPos;
7   in vec4 vColor;
8
9   out vec4 color;
10
11  const int MAX_POINT_LIGHTS = 3;
12  const int MAX_SPOT_LIGHTS = 5;
13
```

ader_light.frag | CommonValues.h

Practica1

```
1   #ifndef COMMONVALS
2   #define COMMONVALS
3   #include "stb_image.h"
4
5   const int MAX_POINT_LIGHTS = 3;
6   const int MAX_SPOT_LIGHTS = 5;
7   #endif
```

```
96  PointLight pointLights[MAX_POINT_LIGHTS];
97  SpotLight spotLights[MAX_SPOT_LIGHTS];
```

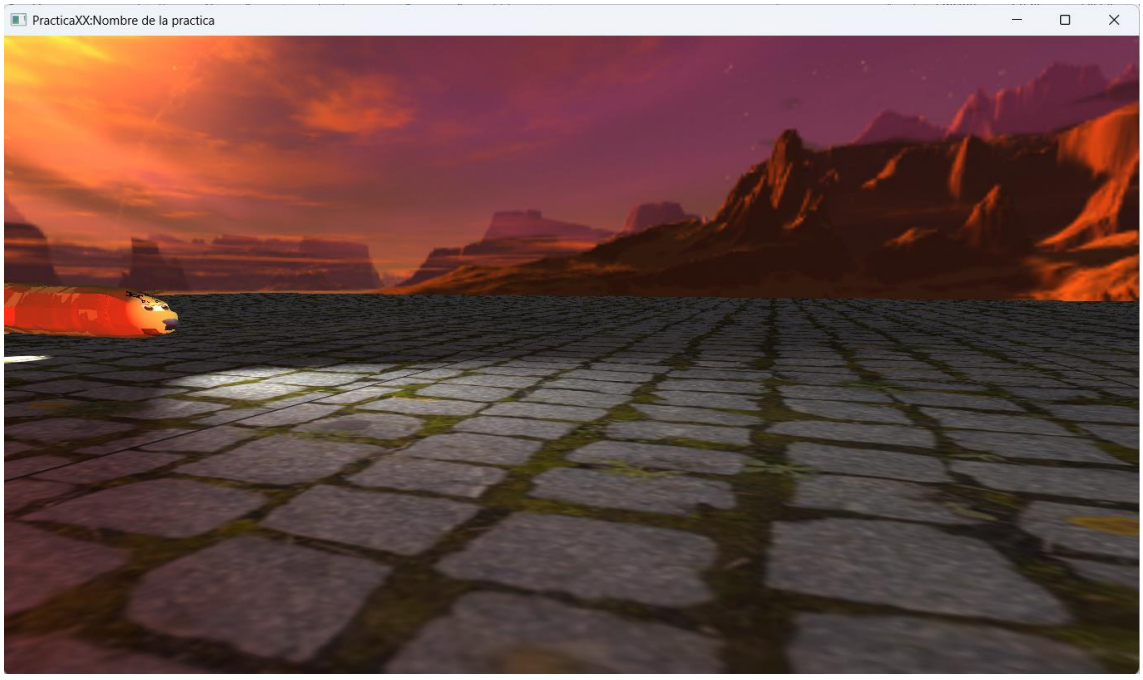
```
579     spotLights[4] = SpotLight(
580         1.0f, 1.0f, 0.9f,
581         1.0f, 4.0f,
582         0.0f, 0.0f, 0.0f,
583         0.0f, 0.0f, 1.0f,
584         1.0f, 0.01f, 0.001f,
585         35.0f
586     );
587     spotLightCount++;
```

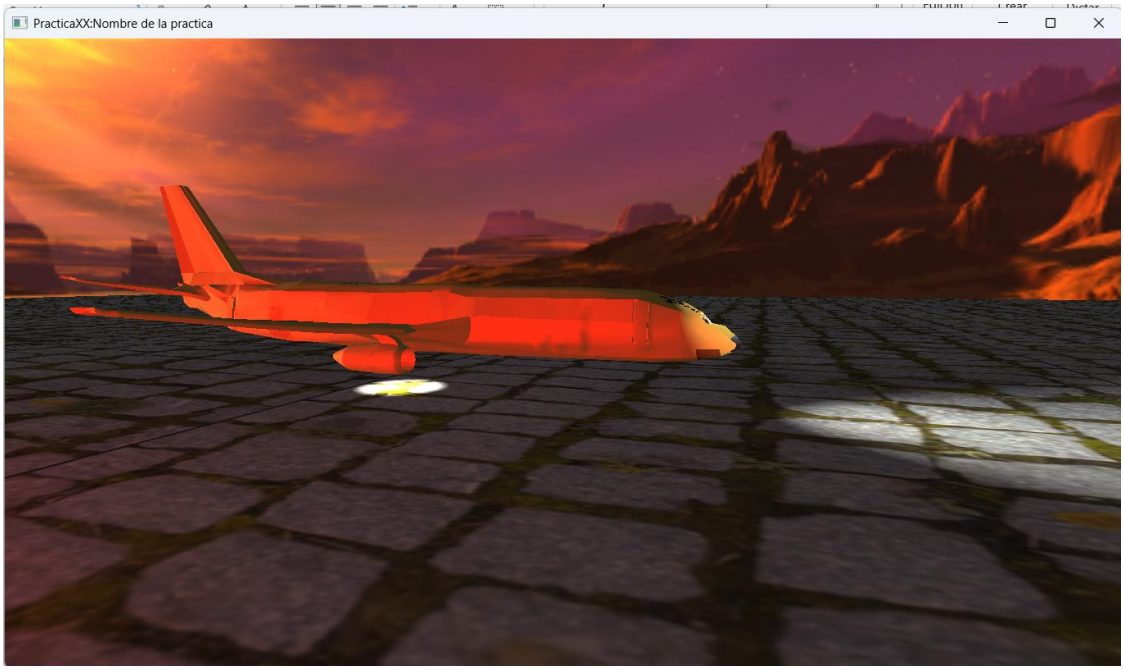
```
917         // Faro 2
918         glm::vec4 posicionLocalFaroFrontal = glm::vec4(0.0f, 0.0f, -2.5f, 1.0f);
919         glm::vec3 posicionMundoFaroFrontal = glm::vec3(model * posicionLocalFaroFrontal);
920         glm::vec4 direccionLocalFaroFrontal = glm::vec4(0.0f, 0.0f, -1.0f, 0.0f);
921         glm::vec3 direccionMundoFaroFrontal = glm::vec3(model * direccionLocalFaroFrontal);
922         direccionMundoFaroFrontal = glm::normalize(direccionMundoFaroFrontal);
923         spotLights[4].SetFlash(posicionMundoFaroFrontal, direccionMundoFaroFrontal);
924         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
925 
926         Avion_M.RenderModel();
```

Capturas de ejecución:




PracticaXX:Nombre de la practica


PracticaXX:Nombre de la practica



En este ejercicio se implementa una luz de faro (*spotlight*) frontal vinculada jerárquicamente al avión, de modo que ilumine hacia adelante y mantenga su posición unida al modelo al avanzar o retroceder en la escena. Para lograr esto, se inicializa el nuevo faro en el índice 4 del arreglo (spotLights[4]) y se actualiza dinámicamente durante el renderizado transformando sus coordenadas y dirección locales mediante la matriz del modelo (model). Asimismo, fue necesario modificar el límite máximo de luces en el código, actualizando la constante MAX_SPOT_LIGHTS a un valor de 5 tanto en el archivo de cabecera CommonValues.h como en el fragment shader shader_light.frag, lo cual permite ampliar la capacidad del arreglo y asegurar que la tarjeta gráfica procese correctamente este nuevo faro frontal.

Capturas de código adicionales:

Practica1 — Window

```
20      void swapBuffers() { return glfwSwapBuffers(mainWindow); }
21      GLfloat getarticulacion1() { return articulacion1; }
22      GLfloat getarticulacion2() { return articulacion2; }
23      GLfloat getarticulacion3() { return articulacion3; }
24      GLfloat getarticulacion4() { return articulacion4; }
25      GLfloat getarticulacion5() { return articulacion5; }
26      GLfloat getarticulacion6() { return articulacion6; }
27      GLfloat getarticulacion7() { return articulacion7; }
28      GLfloat getarticulacion8() { return articulacion8; }
29      GLfloat getarticulacion9() { return articulacion9; }
30      GLfloat getarticulacion10() { return articulacion10; }
31      GLfloat getarticulacion11() { return articulacion11; }
32      GLfloat getarticulacion12() { return articulacion12; }
33      GLfloat getarticulacion13() { return articulacion13; }
34      GLfloat getarticulacion14() { return articulacion14; }
35      GLfloat getarticulacion15() { return articulacion15; }
36      GLfloat getarticulacion16() { return articulacion16; }
37      GLfloat getarticulacion17() { return articulacion17; }
38      GLfloat getarticulacion18() { return articulacion18; }
39      GLfloat getarticulacion19() { return articulacion19; }
40      GLfloat getarticulacion20() { return articulacion20; }
41      GLfloat getarticulacion21() { return articulacion21; }
42      GLfloat getarticulacion22() { return articulacion22; }
43      GLfloat getarticulacion23() { return articulacion23; }
44      GLfloat getarticulacion24() { return articulacion24; }
45      GLfloat getarticulacion25() { return articulacion25; }
46      GLfloat getarticulacion26() { return articulacion26; }
47      GLfloat getarticulacion27() { return articulacion27; }
48      GLfloat getmueveyavion() { return mueveyavion; }
49      GLfloat getmuevexavion() { return muevexavion; }
50
51      ~Window();
52  private:
53      GLFWwindow *mainWindow;
54      GLint width, height;
55      GLfloat articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6,
56          articulacion7, articulacion8, articulacion9, articulacion10, articulacion11, articulacion12, articulacion13,
57          articulacion14, articulacion15, articulacion16, articulacion17, articulacion18, articulacion19, articulacion20,
58          articulacion21, articulacion22, articulacion23, articulacion24, articulacion25, articulacion26, articulacion27;
59      bool keys[1024];
60      GLint bufferWidth, bufferHeight;
61      void createCallbacks();
62      GLfloat lastX;
```

```
46      GLfloat getarticulacion26() { return articulacion26; }
47      GLfloat getarticulacion27() { return articulacion27; }
48      GLfloat getmueveyavion() { return mueveyavion; }
49      GLfloat getmuevexavion() { return muevexavion; }
50
51      ~Window();
52  private:
53      GLFWwindow *mainWindow;
54      GLint width, height;
55      GLfloat articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6,
56          articulacion7, articulacion8, articulacion9, articulacion10, articulacion11, articulacion12, articulacion13,
57          articulacion14, articulacion15, articulacion16, articulacion17, articulacion18, articulacion19, articulacion20,
58          articulacion21, articulacion22, articulacion23, articulacion24, articulacion25, articulacion26, articulacion27;
59      bool keys[1024];
60      GLint bufferWidth, bufferHeight;
61      void createCallbacks();
62      GLfloat lastX;
63      GLfloat lastY;
64      GLfloat xChange;
65      GLfloat yChange;
66      GLfloat muevex;
67      GLfloat mueveyavion;
68      GLfloat muevexavion;
69      bool mouseFirstMoved;
70      static void ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode);
71      static void ManejaMouse(GLFWwindow* window, double xPos, double yPos);
72
73  };
```

```cpp
Window::Window(GLint windowWidth, GLint windowHeight)
{
    width = windowWidth;
    height = windowHeight;
    muevex = 2.0f;
    articulacion1 = 0.0f;
    articulacion2 = 0.0f;
    articulacion3 = 0.0f;
    articulacion4 = 0.0f;
    articulacion5 = 0.0f;
    articulacion6 = 0.0f;
    articulacion7 = 0.0f;
    articulacion8 = 0.0f;
    articulacion9 = 0.0f;
    articulacion10 = 0.0f;
    articulacion11 = 0.0f;
    articulacion12 = 0.0f;
    articulacion13 = 0.0f;
    articulacion14 = 0.0f;
    articulacion15 = 0.0f;
    articulacion16 = 0.0f;
    articulacion17 = 0.0f;
    articulacion18 = 0.0f;
    articulacion19 = 0.0f;
    articulacion20 = 0.0f;
    articulacion21 = 0.0f;
    articulacion22 = 0.0f;
    articulacion23 = 0.0f;
    articulacion24 = 0.0f;
    articulacion25 = 0.0f;
    articulacion26 = 0.0f;
    articulacion27 = 0.0f;
    mueveyavion = 2.0f;
    muevexavion = 2.0f;
```

```cpp

void Window::ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode)
{
    Window* theWindow = static_cast<Window*>(glfwGetWindowUserPointer(window));

    if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
    {
        glfwSetWindowShouldClose(window, GL_TRUE);
    }
    if (key == GLFW_KEY_Y)
    {
        theWindow-> muevex += 1.0;
        theWindow->articulacion7 += 10.0f;
        theWindow->articulacion9 += 10.0f;
        theWindow->articulacion11 += 10.0f;
        theWindow->articulacion13 += 10.0f;
        theWindow->articulacion16 += 10.0f;
        theWindow->articulacion19 += 10.0f;
    }
    if (key == GLFW_KEY_U)
    {
        theWindow-> muevex -= 1.0;
        theWindow->articulacion7 -= 10.0f;
        theWindow->articulacion9 -= 10.0f;
        theWindow->articulacion11 -= 10.0f;
        theWindow->articulacion13 -= 10.0f;
        theWindow->articulacion16 -= 10.0f;
        theWindow->articulacion19 -= 10.0f;
    }


    if (key == GLFW_KEY_Q)
    {
        theWindow->articulacion1 += 10.0f;
    }

    // Y, U, I, O (Movimientos / Articulaciones)
    if (key == GLFW_KEY_1 && (mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion2 += 10.0f;
    }
    if (key == GLFW_KEY_2 && (mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion3 += 10.0f;
    }
    if (key == GLFW_KEY_I && !(mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion4 += 10.0f;
    }
    if (key == GLFW_KEY_O && !(mode & GLFW_MOD_SHIFT))
    {
```

```
175         theWindow->articulacion4 += 10.0f;
176     }
177     if (key == GLFW_KEY_O && !(mode & GLFW_MOD_SHIFT))
178     {
179         theWindow->articulacion5 += 10.0f;
180     }
181
182     // F y G
183     if (key == GLFW_KEY_F)
184     {
185         theWindow->articulacion6 -= 2.0f;
186         if (theWindow->articulacion6 < -45.0f)
187         {
188             theWindow->articulacion6 = -45.0f;
189         }
190     }
191
192     if (key == GLFW_KEY_G)
193     {
194         theWindow->articulacion6 += 2.0f;
195         if (theWindow->articulacion6 > 45.0f)
196         {
197             theWindow->articulacion6 = 45.0f;
198         }
199     }
200
201     if (key == GLFW_KEY_H)
202     {
203         theWindow->articulacion7 += 10.0f;
204     }
205
206     //  J y K
207     if (key == GLFW_KEY_J)
208     {
209         theWindow->articulacion8 -= 2.0f;
210         if (theWindow->articulacion8 < -45.0f)
211         {
212             theWindow->articulacion8 = -45.0f;
213         }
214     }
215
216     if (key == GLFW_KEY_K)
217     {
218         theWindow->articulacion8 += 2.0f;
219         if (theWindow->articulacion8 > 45.0f)
220         {
221             theWindow->articulacion8 = 45.0f;
222         }
223     }
224
225     if (key == GLFW_KEY_L)
```

```
    {
        theWindow->articulacion9 += 10.0f;
    }

    // Z y X (Articulación 10)
    if (key == GLFW_KEY_Z && !(mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion10 -= 2.0f;
        if (theWindow->articulacion10 < -45.0f)
        {
            theWindow->articulacion10 = -45.0f;
        }
    }

    if (key == GLFW_KEY_X && !(mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion10 += 2.0f;
        if (theWindow->articulacion10 > 45.0f)
        {
            theWindow->articulacion10 = 45.0f;
        }
    }

    if (key == GLFW_KEY_C && !(mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion11 += 10.0f;
    }

    // V y B
    if (key == GLFW_KEY_V)
    {
        theWindow->articulacion12 -= 2.0f;
        if (theWindow->articulacion12 < -45.0f)
        {
            theWindow->articulacion12 = -45.0f;
        }
    }

    if (key == GLFW_KEY_B)
    {
        theWindow->articulacion12 += 2.0f;
        if (theWindow->articulacion12 > 45.0f)
        {
            theWindow->articulacion12 = 45.0f;
        }
    }

    if (key == GLFW_KEY_N)
    {
        theWindow->articulacion13 += 10.0f;
    }
```

```
    {
        theWindow->articulacion13 += 10.0f;
    }

    // M y P
    if (key == GLFW_KEY_M)
    {
        theWindow->articulacion14 -= 2.0f;
        if (theWindow->articulacion14 < -45.0f)
        {
            theWindow->articulacion14 = -45.0f;
        }
    }

    if (key == GLFW_KEY_P)
    {
        theWindow->articulacion14 += 2.0f;
        if (theWindow->articulacion14 > 45.0f)
        {
            theWindow->articulacion14 = 45.0f;
        }
    }

    // NÚMEROS (7, 8, 9)
    if (key == GLFW_KEY_7 && !(mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion15 -= 2.0f;
        if (theWindow->articulacion15 < -45.0f)
        {
            theWindow->articulacion15 = -45.0f;
        }
    }

    if (key == GLFW_KEY_8 && !(mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion15 += 2.0f;
        if (theWindow->articulacion15 > 45.0f)
        {
            theWindow->articulacion15 = 45.0f;
        }
    }

    if (key == GLFW_KEY_9 && !(mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion16 += 10.0f;
    }

    // NÚMEROS (1, 2, 3, 4, 5)
    if (key == GLFW_KEY_1 && !(mode & GLFW_MOD_SHIFT))
    {
        theWindow->articulacion17 -= 2.0f;
```

```
321     // NÚMEROS (1, 2, 3, 4, 5)
322     if (key == GLFW_KEY_1 && !(mode & GLFW_MOD_SHIFT))
323     {
324         theWindow->articulacion17 -= 2.0f;
325         if (theWindow->articulacion17 < -45.0f)
326         {
327             theWindow->articulacion17 = -45.0f;
328         }
329     }
330
331     if (key == GLFW_KEY_2 && !(mode & GLFW_MOD_SHIFT))
332     {
333         theWindow->articulacion17 += 2.0f;
334         if (theWindow->articulacion17 > 45.0f)
335         {
336             theWindow->articulacion17 = 45.0f;
337         }
338     }
339
340     if (key == GLFW_KEY_3 && !(mode & GLFW_MOD_SHIFT))
341     {
342         theWindow->articulacion18 -= 2.0f;
343         if (theWindow->articulacion18 < -45.0f)
344         {
345             theWindow->articulacion18 = -45.0f;
346         }
347     }
348
349     if (key == GLFW_KEY_4 && !(mode & GLFW_MOD_SHIFT))
350     {
351         theWindow->articulacion18 += 2.0f;
352         if (theWindow->articulacion18 > 45.0f)
353         {
354             theWindow->articulacion18 = 45.0f;
355         }
356     }
357
358     if (key == GLFW_KEY_5 && !(mode & GLFW_MOD_SHIFT))
359     {
360         theWindow->articulacion19 += 10.0f;
361     }
362
363     if (key == GLFW_KEY_6 && !(mode & GLFW_MOD_SHIFT))
364     {
365         theWindow->mueveyavion += 1.0;
366     }
367     if (key == GLFW_KEY_7 && !(mode & GLFW_MOD_SHIFT))
368     {
369         theWindow->mueveyavion -= 1.0;
```

```
349     if (key == GLFW_KEY_4 && !(mode & GLFW_MOD_SHIFT))
350     {
351         theWindow->articulacion18 += 2.0f;
352         if (theWindow->articulacion18 > 45.0f)
353         {
354             theWindow->articulacion18 = 45.0f;
355         }
356     }
357
358     if (key == GLFW_KEY_5 && !(mode & GLFW_MOD_SHIFT))
359     {
360         theWindow->articulacion19 += 10.0f;
361     }
362
363     if (key == GLFW_KEY_6 && !(mode & GLFW_MOD_SHIFT))
364     {
365         theWindow->mueveyavion += 1.0;
366     }
367     if (key == GLFW_KEY_7 && !(mode & GLFW_MOD_SHIFT))
368     {
369         theWindow->mueveyavion -= 1.0;
370     }
371     if (key == GLFW_KEY_8 && !(mode & GLFW_MOD_SHIFT))
372     {
373         theWindow->muevexavion += 1.0;
374     }
375     if (key == GLFW_KEY_9 && !(mode & GLFW_MOD_SHIFT))
376     {
377         theWindow->muevexavion -= 1.0;
```

Adicionalmente, para gestionar el movimiento interactivo tanto del rover como del avión, se realizaron modificaciones en los archivos window.h y window.cpp. En el archivo window.h, se declararon las variables miembros privadas para las posiciones de desplazamiento (muevevex, mueveyavion y muevevexavion), así como sus respectivos métodos de acceso o getters (getmueveyavion(), getmuevexavion(), entre otros). Por su parte, en window.cpp, estas variables se inicializan con valores base en el constructor de la clase y se actualizan dinámicamente dentro de la función ManejaTeclado al presionar las teclas asignadas, lo que permite controlar de forma fluida el avance y retroceso del rover en el eje X, así como los desplazamientos del avión en la escena.

Video de ejecución: https://www.youtube.com/watch?v=6Tul0zWKVTY

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

El único inconveniente que se presentó durante la realización de estos ejercicios fue el factor tiempo, ya que me costó trabajo lograr que las luces del avión se visualizaran de la manera deseada al configurar los parámetros del reflector (spotLights[3], spotLights[4]) y, específicamente al determinar los valores correctos correspondientes a la atenuación constante, lineal y exponencial (con, lin, exp), así como el ángulo del cono. Estas pruebas de calibración y ajuste realmente me llevaron mucho más tiempo de lo esperado, pero el problema se resolvió probando de forma iterativa hasta alcanzar el mejor efecto visual posible en la escena.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.
b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
c. Conclusión

Tras la realización de esta práctica, considero que con los ejercicios solicitados fue posible comprender de mejor forma los conceptos de iluminación jerárquica y manejo de reflectores (spotlights) mediante código en OpenGL, vinculando las transformaciones de modelos complejos como el avión y el rover. Este proceso permitió conectar la teoría vista en clase con la implementación práctica, facilitando la comprensión de cómo interactúan las matrices de transformación, los arreglos de luces y los shaders dentro de un entorno gráfico tridimensional. Cabe destacar que

el único problema que enfrenté durante el desarrollo fue el factor tiempo, ya que personalmente me tomó bastante más tiempo del esperado lograr la calibración precisa de la atenuación y los ángulos de apertura para que la luz se proyectara de forma óptima. Asimismo, enfrentar y resolver estos detalles técnicos relacionados con la gestión de entradas y la configuración de luces ayudó a consolidar las bases necesarias para desarrollar más prácticas y proyectos futuros.

1. Bibliografía en formato APA

Blender Foundation. (s.f.). Blender Reference Manual. Blender Foundation. Recuperado de https://docs.blender.org/manual/

The Khronos Group. (s.f.). OpenGL 3.3 API Reference Documentation. The Khronos Group. Recuperado de https://www.khronos.org/registry/OpenGL-Refpages/